```
"""
财报分析一体化Pipeline
下载 → 分析 → 存DB → 导出Obsidian
"""

import json
import logging
import psycopg2
from pathlib import Path
from datetime import datetime
from typing import Optional

from .downloader import ReportDownloader
from .analyzer   import ReportAnalyzer
from .exporter   import ObsidianExporter

logger = logging.getLogger(__name__)


class ReportPipeline:
    """
    一体化财报分析流水线

    使用方法:
        from data_layer.report_analyzer import ReportPipeline

        pipeline = ReportPipeline(config={
            'tushare_token': 'xxx',
            'gemini_api_key': 'xxx',
            'gemini_model': 'gemini-1.5-flash',
            'reports_dir': './reports',
            'obsidian_vault': '~/obsidian/fintech',
            'db': {
                'host': 'localhost',
                'port': 5432,
                'dbname': 'fintech',
                'user': 'fintech_user',
                'password': 'xxx',
            }
        })

        # 单只股票单年度
        pipeline.run('600893.SH', 2024)
```

```
    # 批量
    pipeline.batch_run(
        stock_list=['600893.SH', '601021.SH'],
        years=[2023, 2024]
    )
"""

def __init__(self, config: dict):
    self.config = config

    self.downloader = ReportDownloader(
        tushare_token=config['tushare_token'],
        save_dir=config.get('reports_dir', './reports'),
    )
    self.analyzer = ReportAnalyzer(
        api_key=config['gemini_api_key'],
        model=config.get('gemini_model', 'gemini-1.5-flash'),
    )
    self.exporter = ObsidianExporter(
        vault_dir=config.get('obsidian_vault', './obsidian_vault'),
    )
    self._db_config = config.get('db')

# ------------------------------------------------------------------
# 公开接口
# ------------------------------------------------------------------

def run(self, ts_code: str, year: int) -> Optional[dict]:
    """完整流水线：下载 → 分析 → 存档"""
    logger.info(f"━━ Pipeline start: {ts_code} {year} ━━")

    # 1. 下载
    pdf_path = self.downloader.download(ts_code, year)
    if not pdf_path:
        logger.error(f"PDF下载失败，跳过 {ts_code} {year}")
        return None

    # 2. Gemini分析
    result = self.analyzer.analyze(pdf_path)
    if not result:
        logger.error(f"Gemini分析失败，跳过 {ts_code} {year}")
        return None

    # 3. 存入PostgreSQL
    if self._db_config:
        self._save_to_db(ts_code, year, result)
```

```
        # 4. 导出Obsidian
        self.exporter.export(ts_code, year, result)

        logger.info(f"━━ Pipeline done: {ts_code} {year} ━━")
        return result

    def batch_run(
        self,
        stock_list: list[str],
        years: list[int],
    ) -> dict[str, dict]:
        results = {}
        total = len(stock_list) * len(years)
        idx = 0
        for ts_code in stock_list:
            for year in years:
                idx += 1
                logger.info(f"[{idx}/{total}] {ts_code} {year}")
                result = self.run(ts_code, year)
                if result:
                    results[f"{ts_code}_{year}"] = result
        return results

    # ------------------------------------------------------------------
    # 数据库操作
    # ------------------------------------------------------------------

    def _save_to_db(self, ts_code: str, year: int, data: dict):
        """将分析结果存入PostgreSQL"""
        if not self._db_config:
            return
        try:
            conn = psycopg2.connect(**self._db_config)
            cur  = conn.cursor()

            # 确保表存在
            cur.execute("""
                CREATE TABLE IF NOT EXISTS report_analysis (
                    id          SERIAL PRIMARY KEY,
                    ts_code     VARCHAR(20)  NOT NULL,
                    year        INTEGER      NOT NULL,
                    company     VARCHAR(100),
                    raw_json    JSONB        NOT NULL,
                    created_at  TIMESTAMP    DEFAULT NOW(),
                    UNIQUE (ts_code, year)
                )
            """)
```

```
            fin     = data.get("核心财务数据", {})
            company = data.get("基本信息", {}).get("公司名称", "")

            cur.execute("""
                INSERT INTO report_analysis (ts_code, year, company, raw_json)
                VALUES (%s, %s, %s, %s)
                ON CONFLICT (ts_code, year)
                DO UPDATE SET
                    company    = EXCLUDED.company,
                    raw_json   = EXCLUDED.raw_json,
                    created_at = NOW()
            """, (ts_code, year, company, json.dumps(data, ensure_ascii=False)))

            conn.commit()
            cur.close()
            conn.close()
            logger.info(f"[DB] {ts_code} {year} 已存入PostgreSQL")

        except Exception as e:
            logger.error(f"[DB] 写入失败: {e}")

    def query_from_db(self, ts_code: str, year: int) -> Optional[dict]:
        """从DB读取分析结果"""
        if not self._db_config:
            return None
        try:
            conn = psycopg2.connect(**self._db_config)
            cur  = conn.cursor()
            cur.execute(
                "SELECT raw_json FROM report_analysis WHERE ts_code=%s AND year=%
                (ts_code, year)
            )
            row = cur.fetchone()
            cur.close()
            conn.close()
            return row[0] if row else None
        except Exception as e:
            logger.error(f"[DB] 查询失败: {e}")
            return None
```